

Aplicación de Escritorio Java JDBC + DAO + Swing

Introducción

En este documento podrás encontrar toda la documentación referida a esta aplicación de escritorio.

En ella manejamos una base de datos de la cafetería del IES Francisco Ayala, donde podremos almacenar clientes, productos y pedidos. Tendremos dos formas de acceder a ella, mediante consola con el archivo Main.java o mediante vistas (Swing) con el archivo MainJSwing.java

Encontraremos organizado el proyecto de la siguiente forma:

lib/

└── (librerías externas del proyecto)

src/

└── dao/ ← DAO e implementaciones DAO

├── ClienteDAO.java
├── ClienteDAOImpl.java
├── PedidoDAO.java
├── PedidoDAOImpl.java
├── PedidoProductoDAO.java
├── PedidoProductoDAOImpl.java
├── ProductoDAO.java
└── ProductoDAOImpl.java

└── model/ □ Capa Modelo

├── Cliente.java
├── Pedido.java
├── Producto.java
└── TipoCliente.java

└── db/ □ Conexión a la base de datos

└── ConexionDB.java

view/ ← Vistas (Swing)

└── cliente/

├── AltaClientePanel.java
├── ClientesPanel.java
└── EditarClientePanel.java

└── comun/

├── AbstractCatalogoProductosPanel.java
└── AbstractPanelBotones.java

└── pedidos/

└── NuevoPedidoPanel.java

```
| -- PedidoDetallePanel.java
| | -- PedidosListaPanel.java
|
|-- principal/
| | -- MainPanel.java
|
|-- producto/
|   -- AltaProductoPanel.java
|   -- EditarProductoPanel.java
|   |-- ProductosPanel.java
```

documentacion/ ← Documentación en diferentes tipos de archivos

```
-- README.md
-- documentacion.pdf
|-- documentacion.odt
```

initdb.sql ← Lista de todas las consultas utilizadas para la base de datos

Main.java ← Main por consola(VSC)

MainJ Swing.java ← Main Swing

docker-compose.yml ← Crea una serie de servicios Docker para poder iniciar la aplicación

Base de datos

Base de datos: cafeteria_ayala | Host: localhost:3308

Toda la información de la base de datos como tablas y datos están almacenados en initdb.sql para poder ejecutar las consultas.

Si quisiéramos verlo en la consola del propio ordenador con el siguiente comando podremos mostrar por pantalla toda la base de datos en formato SQL:

```
docker exec mysql-java-server /usr/bin/mysqldump -u root --password=admin
cafeteria_ayala
```

ConexionDB.java

-Proporciona la conexión MySQL mediante DriverManager.

-Requiere el conector MySQL (jar) en Referenced Libraries.

Capa de Modelo

Para la parte de modelo hay 8 archivos, 4 de ellos enums que acompañan a los demás. En cada uno están declaradas las variables, los constructores y los getters and setters.

¿Por qué hay 3 constructores?

Se crean 3 constructores porque cada uno permite crear objetos de una forma distinta dependiendo de la necesidad del programa.

Un constructor permite crear el cliente sin ID, porque la base de datos lo genera automáticamente(incrementando), otro crea el objeto con todos los datos y el vacío permite asignarlos más tarde con setters.

Clases principales:

- Cliente → Tiene id, nombre y tipo (ALUMNO / PROFESOR)
- Producto → Tiene id, nombre, precio, cantidad y tipo (COMIDA / BEBIDA)
- Pedido → Tiene id, precio total, método de pago, estado y cliente asociado (clienteID)
- PedidoProducto → Relación entre un pedido y sus productos (cantidad incluida)

Enumeraciones:

- TipoCliente → ALUMNO, PROFESOR
- TipoProducto → COMIDA, BEBIDA
- Estado → ABIERTO, CERRADO
- MetodoPago → EFECTIVO, TARJETA, BIZUM

Capa DAO e Implementaciones

Arquitectura DAO (Data Access Object) que separa la lógica de acceso a base de datos del resto de la aplicación. Cada entidad tiene una interfaz y su implementación.

Clases principales:

- Cliente → id, nombre, tipo (ALUMNO / PROFESOR)
- Producto → id, nombre, precio, cantidad, tipo (COMIDA / BEBIDA)
- Pedido → id, precio total, método de pago, estado, cliente asociado
- PedidoProducto → relación entre pedido y producto con cantidad

Enumeraciones:

- TipoCliente → ALUMNO, PROFESOR
- TipoProducto → COMIDA, BEBIDA
- Estado → ABIERTO, CERRADO
- MetodoPago → EFECTIVO, TARJETA, BIZUM

ClienteDAO / ClienteDAOImpl

- añadirCliente(Cliente) → INSERT en tabla cliente
- borrarCliente(int id) → DELETE por id
- buscarCliente(String nombre) → SELECT por nombre
- listarClientes() → SELECT todos

ProductoDAO / ProductoDAOImpl

- añadirProducto(Producto) → INSERT en tabla producto
- borrarProducto(int id) → DELETE por id
- modificarProducto(Producto) → UPDATE por id
- buscarProducto(int id) → SELECT por id (lanza Exception si no existe)
- listarProductos() → SELECT todos

PedidoDAO / PedidoDAOImpl

- añadirPedido(Pedido) → INSERT en tabla pedido (sin metodo_pago por ahora)
- borrarPedido(int id) → DELETE por id
- modificarPedido(Pedido) → UPDATE por id (incluye metodo_pago y precio_total)
- buscarPedido(String nombre) → SELECT por nombre
- listarPedidos() → SELECT todos, calcula precio sumando productos

PedidoProductoDAO / PedidoProductoDAOImpl

- añadirProducto(int pedidoid, int productoid, int cantidad)
- INSERT en tabla pedido_producto

Swing / vistas

Interfaz gráfica desarrollada en Java Swing con navegación mediante CardLayout (sistema de "tarjetas" que se muestran y ocultan sin abrir nuevas ventanas).

view/

-- principal/	
→ MainJPanel.java	↳ Menú principal
-- comun/	
-- AbstractPanelBotones.java	→ Panel base reutilizable
-- AbstractCatalogoProductosPanel.java	→ Catálogo de productos reutilizable
-- cliente/	
-- ClientesJPanel.java	→ Lista de clientes
-- AltaClientePanel.java	→ Formulario nuevo cliente
-- EditarClientePanel.java	→ Formulario editar cliente
-- producto/	
-- ProductosJPanel.java	→ Catálogo de productos
-- AltaProductoPanel.java	→ Formulario nuevo producto
-- EditarProductoPanel.java	→ Formulario editar producto
-- pedidos/	
-- PedidosListaPanel.java	→ Lista de pedidos abiertos
-- NuevoPedidoPanel.java	→ Formulario nuevo pedido
-- PedidoDetallePanel.java	→ Detalle y cobro de pedido

CLASES COMUNES (view/comun)

AbstractPanelBotones

Panel base del que heredan ClientesJPanel, ProductosJPanel y MainJPanel. Proporciona:

- Cabecera con título, botón "Menu principal" y botones "Añadir" / "Actualizar" (opcionales).
- Rejilla de 4 columnas con botones generados desde getBotones().
- Método refreshGrid() para recargar la rejilla.

Métodos a sobreescribir por las subclases:

- getTitulo() → texto del título
- getBotones() → lista de etiquetas para la rejilla
- añadirRegistro() → acción del botón Añadir
- actualizarRegistro() → acción del botón Actualizar
- mostrarBotonAñadir() → si es false, oculta Añadir y Actualizar

AbstractCatalogoProductosPanel

Panel reutilizable que muestra productos agrupados por tipo (Comida / Bebida) en una rejilla de botones.

Usado tanto en ProductosJPanel como en PedidoDetallePanel.

- refrescarCatalogo(List<Producto>) → repinta la rejilla
- alSeleccionarProducto(Producto) → acción al pulsar un producto (abstracto, lo define cada uso)
- usarRejillaAdaptableAncho() → si true, recalcula columnas al redimensionar la ventana

MENÚ PRINCIPAL (view/principal)

MainJPanel

Hereda de AbstractPanelBotones. Muestra tres botones:

- Clientes
- Productos
- Pedidos

No tiene botones Añadir / Actualizar.

SECCIÓN CLIENTES (view/cliente)

ClientesJPanel

Lista todos los clientes como botones en una rejilla.

- Añadir → navega a AltaClientePanel
- Actualizar → navega a EditarClientePanel

AltaClientePanel

Formulario para crear un nuevo cliente.

Campos: Nombre (texto), Tipo (combo: ALUMNO / PROFESOR)

- Guardar → inserta en BD y vuelve a la lista
- Cancelar → vuelve a la lista sin guardar
- Se limpia automáticamente al mostrarse

EditarClientePanel

Formulario para modificar un cliente existente.

- Combo superior para seleccionar el cliente a editar.
- Al seleccionar, rellena los campos con sus datos actuales.

Campos editables: Nombre, Tipo

- Guardar → actualiza en BD y vuelve a la lista
- Cancelar → vuelve a la lista sin guardar

SECCIÓN PRODUCTOS (view/producto)

ProductosJPanel

Muestra el catálogo de productos usando

AbstractCatalogoProductosPanel (agrupado por Comida / Bebida).

- Añadir → navega a AltaProductoPanel
- Actualizar → navega a EditarProductoPanel

AltaProductoPanel

Formulario para crear un nuevo producto.

Campos: Nombre, Cantidad, Precio, Tipo (COMIDA / BEBIDA)

Validaciones: nombre no vacío, cantidad ≥ 0 , precio ≥ 0

- Guardar → inserta en BD y vuelve al catálogo
- Cancelar → vuelve sin guardar

EditarProductoPanel

Formulario para modificar un producto existente.

- Combo superior para seleccionar el producto (muestra id – nombre).
- Al seleccionar, rellena los campos con sus datos actuales.

Campos editables: Nombre, Cantidad, Precio, Tipo

Mismas validaciones que AltaProductoPanel.

- Guardar → actualiza en BD y vuelve al catálogo
- Cancelar → vuelve sin guardar

SECCIÓN PEDIDOS (view/pedidos)

PedidosListaPanel

Muestra la lista de pedidos con estado ABIERTO.

Cada pedido aparece como un botón con formato:

"Pedido #id – NombreCliente – precio €"

- Nuevo pedido → navega a NuevoPedidoPanel
- Al pulsar un pedido → abre PedidoDetallePanel para ese pedido
- Se refresca automáticamente al mostrarse

NuevoPedidoPanel

Formulario para crear un nuevo pedido.

- Combo para seleccionar el cliente.
- Al confirmar: crea el pedido en BD con estado ABIERTO y navega directamente a PedidoDetallePanel.
- Cancelar → vuelve a la lista de pedidos

PedidoDetallePanel

Panel principal de gestión de un pedido. Tiene dos modos internos que alternan sin cambiar de tarjeta:

MODO EDITAR:

- Catálogo de productos (izquierda) para añadir líneas.
- Tabla de líneas del pedido con columnas:
Producto | Cantidad | P. unitario | Subtotal
- Total del pedido actualizado en tiempo real.
- Botón "Cerrar pedido" → pasa al modo cobro.

MODO COBRO (solo efectivo):

- Muestra el total a pagar.
- Campo para introducir el importe entregado.
- Calcula y muestra el cambio en tiempo real.
- Confirmar → cierra el pedido en BD con MetodoPago.EFECTIVO y vuelve a la lista de pedidos.
- Volver al pedido → regresa al modo editar.